Topic 13 ANN Perceptron Learning Rules — Complete Study Guide

Let me break down the entire lecture clearly so your concepts are solid and you can handle any scenario-based math.

The Big Picture: What's Happening?

A perceptron is a single neuron. It takes inputs, multiplies them by weights, and produces an output. The goal of **learning/training** is to adjust the weights so the output matches the desired output. The general update rule for ALL three methods is:

$w(k) = w(k-1) + \Delta w(k)$

The only difference between the three rules is **how Δw is calculated**.

Understanding the Indices (Important for Math)

- **i = 1 to m** → inputs (x_1, x_2, ... x_m)
- **j = 1 to n** → nodes/neurons
- **k = 1 to p** → training patterns/sequences

So x(k) means the k-th training input vector, d(k) means the desired output for that pattern.

Rule 1: Perceptron Learning Rule

Concept

The perceptron uses a **hard-limiting (sign) activation function**:

- $y = \text{sgn}(w^T x)$ → output is either +1 or −1
- The learning signal $r = d - y$ (difference between desired and actual)

Formula

$$\Delta w = \eta(d - y)x$$

Where η is the learning rate (constant), d is desired output, y is actual output, x is the input vector.

Since d takes values ±1, this simplifies to:

- If $y \neq d \rightarrow \Delta w = 2\eta d \cdot x$
- If $y = d \rightarrow \Delta w = 0$ (no update needed)

Step-by-Step Process

Given: initial weights w(1), learning rate η, training pairs {x(k), d(k)}

For each training pattern k:

1. Compute net input: $\mathbf{I(k) = w^T(k) \cdot x(k)}$ (dot product)
2. Compute output: $\mathbf{y(k) = sgn(I(k))} \rightarrow +1$ if $I > 0$, -1 if $I < 0$
3. Compute learning signal: $\mathbf{r(k) = d(k) - y(k)}$
4. Update weights: $\mathbf{w(k+1) = w(k) + \eta \cdot r(k) \cdot x(k)}$

Worked Example (from slide)

Given:

- $w(1) = [1, -1, 0, 0.5]^T$
- $\eta = 0.1$
- $x(1) = [1, -2, 0, 1]^T$, $d(1) = -1$
- $x(2) = [0, 1.5, -0.5, 1]^T$, $d(2) = -1$
- $x(3) = [-1, 1, 0.5, 1]^T$, $d(3) = 1$

Step 1 (k=1):

$I(1) = [1, -1, 0, 0.5] \cdot [1, -2, 0, 1] = (1\times1) + (-1\times-2) + (0\times0) + (0.5\times1) = 1 + 2 + 0 + 0.5 = \mathbf{3.5}$

$y(1) = sgn(3.5) = \mathbf{+1}$

$r(1) = d(1) - y(1) = -1 - 1 = \mathbf{-2}$

$\Delta w(1) = \eta \cdot r(1) \cdot x(1) = 0.1 \times (-2) \times [1, -2, 0, 1]^T = -0.2 \times [1, -2, 0, 1]^T = [-0.2, 0.4, 0, -0.2]^T$

$w(2) = w(1) + \Delta w(1) = [1, -1, 0, 0.5]^T + [-0.2, 0.4, 0, -0.2]^T = \mathbf{[0.8, -0.6, 0, 0.3]^T}$

Step 2 (k=2):

$I(2) = [0.8, -0.6, 0, 0.3] \cdot [0, 1.5, -0.5, 1] = 0 + (-0.9) + 0 + 0.3 = \mathbf{-0.6}$

y(2) = sgn(−0.6) = **−1**

r(2) = d(2) − y(2) = −1 − (−1) = **0**

Since r = 0 → **Δw = 0**, so **w(3) = w(2) = [0.8, −0.6, 0, 0.3]ᵀ**

Step 3 (k=3):

I(3) = [0.8, −0.6, 0, 0.3] · [−1, 1, 0.5, 1] = −0.8 + (−0.6) + 0 + 0.3 = **−1.1**

y(3) = sgn(−1.1) = **−1**

r(3) = d(3) − y(3) = 1 − (−1) = **+2**

Δw(3) = 0.1 × 2 × [−1, 1, 0.5, 1]ᵀ = 0.2 × [−1, 1, 0.5, 1]ᵀ = [−0.2, 0.2, 0.1, 0.2]ᵀ

w(4) = [0.8, −0.6, 0, 0.3]ᵀ + [−0.2, 0.2, 0.1, 0.2]ᵀ = **[0.6, −0.4, 0.1, 0.5]ᵀ**

This matches the slide's final answer ✓

Rule 2: ADALINE Learning Rule (Widrow-Hoff / LMS Rule)

Concept

ADALINE uses a **linear activation function** — y = I = wᵀx directly (no sgn). The goal is to minimize the cost (error) function using **Gradient Descent**.

Cost function:

$$E(w) = \tfrac{1}{2} \Sigma_{\square} (d(k) - y(k))^2 = \tfrac{1}{2} \Sigma_{\square} (d(k) - w^T x(k))^2$$

To minimize E, move weights in the direction of the **negative gradient**.

Formula

$$\Delta w(k) = \eta(d(k) - y(k)) \cdot x(k)$$

Here y(k) = wᵀ(k) · x(k) (just the raw dot product, no activation function applied). So r = d(k) − wᵀ(k)x(k).

Key Difference from Perceptron

In Perceptron, y = sgn(wᵀx). In ADALINE, y = wᵀx (the raw number). The weight update formula *looks* the same but y is computed differently — this is the critical distinction.

ADALINE Learning Rule — Complete Worked Example

Key reminder: In ADALINE, $y = w^T x$ (raw dot product, NO sgn function). The error uses this raw value.

Formula: $\Delta w(k) = \eta \cdot (d(k) - y(k)) \cdot x(k)$

Given

- $w(1) = [1, -1, 0, 0.5]^T$
- $\eta = 0.1$
- $x(1) = [1, -2, 0, 1]^T$, $d(1) = -1$
- $x(2) = [0, 1.5, -0.5, 1]^T$, $d(2) = -1$
- $x(3) = [-1, 1, 0.5, 1]^T$, $d(3) = 1$

Step 1 (k = 1)

Compute y(1):

$y(1) = w^T(1) \cdot x(1) = (1\times1) + (-1\times-2) + (0\times0) + (0.5\times1) = 1 + 2 + 0 + 0.5 =$ **3.5**

(Note: In ADALINE, y = 3.5 directly. We do NOT apply sgn.)

Compute error:

$d(1) - y(1) = -1 - 3.5 =$ **−4.5**

Compute Δw(1):

$\Delta w(1) = \eta \cdot (d-y) \cdot x(1) = 0.1 \times (-4.5) \times [1, -2, 0, 1]^T = -0.45 \times [1, -2, 0, 1]^T$

$$\Delta w(1) = \begin{bmatrix} -0.45 \times 1 \\ -0.45 \times -2 \\ -0.45 \times 0 \\ -0.45 \times 1 \end{bmatrix} = \begin{bmatrix} -0.45 \\ 0.90 \\ 0.00 \\ -0.45 \end{bmatrix}$$

Update weights:

$w(2) = w(1) + \Delta w(1)$

$$\begin{bmatrix} 1 \\ \end{bmatrix} \begin{bmatrix} -0.45 \end{bmatrix} \begin{bmatrix} 0.55 \end{bmatrix}$$

$$= \begin{bmatrix} -1 \\ \end{bmatrix} + \begin{bmatrix} 0.90 \end{bmatrix} = \begin{bmatrix} -0.10 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ \end{bmatrix} \begin{bmatrix} 0.00 \end{bmatrix} \begin{bmatrix} 0.00 \end{bmatrix}$$

$$\begin{bmatrix} 0.5 \\ \end{bmatrix} \begin{bmatrix} -0.45 \end{bmatrix} \begin{bmatrix} 0.05 \end{bmatrix}$$

$w(2) = [0.55, -0.10, 0, 0.05]^T$

Step 2 (k = 2)

Compute y(2):

$y(2) = w^T(2) \cdot x(2) = (0.55 \times 0) + (-0.10 \times 1.5) + (0 \times -0.5) + (0.05 \times 1) = 0 + (-0.15) + 0 + 0.05 =$ **-0.10**

Compute error:

$d(2) - y(2) = -1 - (-0.10) =$ **-0.90**

Compute Δw(2):

$\Delta w(2) = 0.1 \times (-0.90) \times [0, 1.5, -0.5, 1]^T = -0.09 \times [0, 1.5, -0.5, 1]^T$

$$\Delta w(2) = \begin{bmatrix} -0.09 \times 0 \\ \end{bmatrix} \begin{bmatrix} 0.000 \end{bmatrix}$$

$$\begin{bmatrix} -0.09 \times 1.5 \end{bmatrix} \begin{bmatrix} -0.135 \end{bmatrix}$$

$$\begin{bmatrix} -0.09 \times -0.5 \end{bmatrix} \begin{bmatrix} 0.045 \end{bmatrix}$$

$$\begin{bmatrix} -0.09 \times 1 \\ \end{bmatrix} \begin{bmatrix} -0.090 \end{bmatrix}$$

Update weights:

$w(3) = w(2) + \Delta w(2)$

$$\begin{bmatrix} 0.55 \\ \end{bmatrix} \begin{bmatrix} 0.000 \end{bmatrix} \begin{bmatrix} 0.550 \end{bmatrix}$$

$$= \begin{bmatrix} -0.10 \\ \end{bmatrix} + \begin{bmatrix} -0.135 \end{bmatrix} = \begin{bmatrix} -0.235 \end{bmatrix}$$

$$\begin{bmatrix} 0.00 \\ \end{bmatrix} \begin{bmatrix} 0.045 \end{bmatrix} \begin{bmatrix} 0.045 \end{bmatrix}$$

$$\begin{bmatrix} 0.05 \\ \end{bmatrix} \begin{bmatrix} -0.090 \end{bmatrix} \begin{bmatrix} -0.040 \end{bmatrix}$$

$w(3) = [0.550, -0.235, 0.045, -0.040]^T$

Step 3 (k = 3)

Compute y(3):

$y(3) = w^T(3) \cdot x(3) = (0.550 \times -1) + (-0.235 \times 1) + (0.045 \times 0.5) + (-0.040 \times 1) = -0.550 + (-0.235) + 0.0225 + (-0.040) =$ **-0.8025**

Compute error:

$d(3) - y(3) = 1 - (-0.8025) =$ **$+1.8025$**

Compute Δw(3):

$\Delta w(3) = 0.1 \times (1.8025) \times [-1, 1, 0.5, 1]^T = 0.18025 \times [-1, 1, 0.5, 1]^T$

$$\Delta w(3) = \begin{bmatrix} 0.18025 \times -1 \\ 0.18025 \times 1 \\ 0.18025 \times 0.5 \\ 0.18025 \times 1 \end{bmatrix} \begin{bmatrix} -0.1803 \\ 0.1803 \\ 0.0901 \\ 0.1803 \end{bmatrix}$$

Update weights:

$w(4) = w(3) + \Delta w(3)$

$$\begin{bmatrix} 0.550 \\ -0.235 \\ 0.045 \\ -0.040 \end{bmatrix} \begin{bmatrix} -0.1803 \\ 0.1803 \\ 0.0901 \\ 0.1803 \end{bmatrix} \begin{bmatrix} 0.3697 \\ -0.0547 \\ 0.1351 \\ 0.1403 \end{bmatrix}$$

$w(4) = [0.3697, -0.0547, 0.1351, 0.1403]^T$

Full Summary Table

Step	w used	I = $w^T x$	y (raw)	d	d−y	Δw	w (updated)

k=1	w(1)	3.5	3.5	−1	−4.5	[−0.45, 0.90, 0, −0.45]	w(2) = [0.55, −0.10, 0, 0.05]
k=2	w(2)	−0.10	−0.10	−1	−0.90	[0, −0.135, 0.045, −0.09]	w(3) = [0.55, −0.235, 0.045, −0.04]
k=3	w(3)	−0.8025	−0.8025	+1	+1.8025	[−0.1803, 0.1803, 0.0901, 0.1803]	w(4) = [0.3697, −0.0547, 0.1351, 0.1403]

Rule 3: Delta Learning Rule

Concept

Delta rule is used when the activation function is **continuous and differentiable** (e.g., sigmoid). It also uses gradient descent but now the derivative of the activation function appears in the weight update.

Formula

$$\Delta w(k) = \eta \cdot (d(k) - y(k)) \cdot \Phi'(I(k)) \cdot x(k)$$

Where:

- y(k) = Φ(I(k)) — sigmoid output
- Φ'(I(k)) — derivative of the activation function

Activation Functions and Their Derivatives

Unipolar Sigmoid: (output between 0 and 1)

- Φ(I) = 1 / (1 + e^(−I))
- Φ'(I) = y(1 − y) ← very clean formula!

Bipolar Sigmoid: (output between −1 and +1)

- Φ(I) = 2/(1 + e^(−I)) − 1
- Φ'(I) = ½(1 − y²)

Delta Learning Rule — Complete Worked Example

Formula: $\Delta w(k) = \eta \cdot (d(k) - y(k)) \cdot \Phi'(I(k)) \cdot x(k)$

Activation (Unipolar Sigmoid):

- $y = \Phi(I) = 1/(1 + e^{(-I)})$
- $\Phi'(I) = y(1 - y)$

Given

- $w(1) = [1, -1, 0, 0.5]^T$
- $\eta = 0.1$
- $x(1) = [1, -2, 0, 1]^T$, $d(1) = -1$
- $x(2) = [0, 1.5, -0.5, 1]^T$, $d(2) = -1$
- $x(3) = [-1, 1, 0.5, 1]^T$, $d(3) = 1$

Step 1 (k = 1)

Compute I(1):

$I(1) = w^T(1) \cdot x(1) = (1\times1) + (-1\times-2) + (0\times0) + (0.5\times1) = 1 + 2 + 0 + 0.5 =$ **3.5**

Compute y(1):

$y(1) = 1/(1 + e^{(-3.5)}) = 1/(1 + 0.0302) = 1/1.0302 =$ **0.9704**

Compute Φ'(I(1)):

$\Phi'(I(1)) = y(1) \times (1 - y(1)) = 0.9704 \times (1 - 0.9704) = 0.9704 \times 0.0296 =$ **0.0287**

Compute error:

$d(1) - y(1) = -1 - 0.9704 =$ **−1.9704**

Compute Δw(1):

$\Delta w(1) = \eta \times (d-y) \times \Phi' \times x(1) = 0.1 \times (-1.9704) \times 0.0287 \times [1, -2, 0, 1]^T = 0.1 \times (-0.0566) \times [1, -2, 0, 1]^T = -0.00566 \times [1, -2, 0, 1]^T$

$\Delta w(1) = [-0.00566 \times 1 \;\;] \quad [-0.0057]$

[−0.00566 × −2] [0.0113]

[−0.00566 × 0] [0.0000]

[−0.00566 × 1] [−0.0057]

Update weights:

w(2) = w(1) + Δw(1)

[1.0] [−0.0057] [0.9943]

= [−1.0] + [0.0113] = [−0.9887]

[0.0] [0.0000] [0.0000]

[0.5] [−0.0057] [0.4943]

w(2) = [0.9943, −0.9887, 0.0000, 0.4943]^T

Step 2 (k = 2)

Compute I(2):

I(2) = w^T(2) · x(2) = (0.9943×0) + (−0.9887×1.5) + (0.0000×−0.5) + (0.4943×1) = 0 + (−1.4831) + 0 + 0.4943 = **−0.9888**

Compute y(2):

y(2) = 1/(1 + e^(0.9888)) = 1/(1 + 2.6879) = 1/3.6879 = **0.2712**

Compute Φ'(I(2)):

Φ'(I(2)) = y(2) × (1 − y(2)) = 0.2712 × (1 − 0.2712) = 0.2712 × 0.7288 = **0.1977**

Compute error:

d(2) − y(2) = −1 − 0.2712 = **−1.2712**

Compute Δw(2):

Δw(2) = 0.1 × (−1.2712) × 0.1977 × [0, 1.5, −0.5, 1]^T = 0.1 × (−0.2513) × [0, 1.5, −0.5, 1]^T = −0.02513 × [0, 1.5, −0.5, 1]^T

$$\Delta w(2) = [-0.02513 \times 0\ ]\ \ [\ 0.0000]$$
$$[-0.02513 \times 1.5]\ \ [-0.0377]$$
$$[-0.02513 \times -0.5]\ \ [\ 0.0126]$$
$$[-0.02513 \times 1\ ]\ \ [-0.0251]$$

Update weights:

$$w(3) = w(2) + \Delta w(2)$$

$$\begin{bmatrix} 0.9943 \\ 0.0000 \\ 0.4943 \end{bmatrix} + \begin{bmatrix} 0.0000 \\ 0.0126 \\ -0.0251 \end{bmatrix} = \begin{bmatrix} 0.9943 \\ -1.0264 \\ 0.0126 \\ 0.4692 \end{bmatrix}$$

$w(3) = [0.9943, -1.0264, 0.0126, 0.4692]^T$

Step 3 (k = 3)

Compute I(3):

$I(3) = w^T(3) \cdot x(3) = (0.9943 \times -1) + (-1.0264 \times 1) + (0.0126 \times 0.5) + (0.4692 \times 1) = -0.9943 + (-1.0264) + 0.0063 + 0.4692 =$ **-1.5452**

Compute y(3):

$y(3) = 1/(1 + e^{(1.5452)}) = 1/(1 + 4.6892) = 1/5.6892 =$ **0.1758**

Compute Φ'(I(3)):

$\Phi'(I(3)) = y(3) \times (1 - y(3)) = 0.1758 \times (1 - 0.1758) = 0.1758 \times 0.8242 =$ **0.1449**

Compute error:

$d(3) - y(3) = 1 - 0.1758 =$ **+0.8242**

Compute Δw(3):

$\Delta w(3) = 0.1 \times (0.8242) \times 0.1449 \times [-1, 1, 0.5, 1]^T = 0.1 \times 0.1194 \times [-1, 1, 0.5, 1]^T = 0.01194 \times [-1, 1, 0.5, 1]^T$

$$\Delta w(3) = \begin{bmatrix} 0.01194 \times -1 \\ 0.01194 \times 1 \\ 0.01194 \times 0.5 \\ 0.01194 \times 1 \end{bmatrix} \begin{bmatrix} -0.0119 \\ 0.0119 \\ 0.0060 \\ 0.0119 \end{bmatrix}$$

Update weights:

$w(4) = w(3) + \Delta w(3)$

$$= \begin{bmatrix} 0.9943 \\ -1.0264 \\ 0.0126 \\ 0.4692 \end{bmatrix} + \begin{bmatrix} -0.0119 \\ 0.0119 \\ 0.0060 \\ 0.0119 \end{bmatrix} = \begin{bmatrix} 0.9824 \\ -1.0145 \\ 0.0186 \\ 0.4811 \end{bmatrix}$$

$w(4) = [0.9824, -1.0145, 0.0186, 0.4811]^T$

Full Summary Table

Step	$I = w^T x$	$y = \Phi(I)$	$d - y$	$\Phi' = y(1-y)$	Δw scale ($\eta \times err \times \Phi'$)	w (updated)
k=1	3.5	0.9704	−1.9704	0.0287	−0.00566	w(2) = [0.9943, −0.9887, 0, 0.4943]
k=2	−0.9888	0.2712	−1.2712	0.1977	−0.02513	w(3) = [0.9943, −1.0264, 0.0126, 0.4692]

k=3	−1.5452	0.1758	+0.8242	0.1449	+0.01194	w(4) = [0.9824, −1.0145, 0.0186, 0.4811]

1-Line Interpretation

Delta rule corrects weights using both the **error magnitude** and the **slope of the sigmoid** (Φ'), so when output is near saturation (close to 0 or 1), corrections are naturally smaller — making learning more careful and stable than ADALINE or Perceptron.

Summary Table — The 3 Rules Compared

	Perceptron	**ADALINE**	**Delta**
Activation	Hard-limiting (sgn)	Linear (y = I)	Sigmoid Φ(I)
y =	sgn(w^Tx) = ±1	w^Tx (raw number)	Φ(w^Tx) ∈ (0,1)
Δw =	η(d−y)x	η(d−y)x	η(d−y)Φ'(I)x
Extra term?	No	No	Yes: Φ'(I)
Update when?	Only if y ≠ d	Always	Always

How to Handle Scenario-Based Questions

When you see a 4-5 line scenario in your exam, extract these things:

Checklist:

1. How many inputs? → size of x vector
2. What are the input values? → fill x(k)
3. What is the desired output d(k)?
4. What are the initial weights w(1)?
5. What is η (learning rate)?
6. Which rule? → Look for keywords:
    - "sign function" / "hard-limiting" → **Perceptron**
    - "linear" / "LMS" / "Widrow-Hoff" → **ADALINE**
    - "sigmoid" / "differentiable" → **Delta**
7. How many iterations?

Then follow the steps:

- Compute I = $w^T x$ (always same)
- Apply activation (rule-specific)
- Compute error = d − y
- Compute Δw (rule-specific)
- Update w

Quick Reference Formulas (Write These in Your Formula Sheet)

General: $w(k+1) = w(k) + \Delta w(k)$

Perceptron: $\Delta w = \eta(d - sgn(w^T x)) \cdot x$
→ update only when output is wrong

ADALINE: $\Delta w = \eta(d - w^T x) \cdot x$
→ y is the raw dot product

Delta: $\Delta w = \eta(d - \Phi(w^T x)) \cdot \Phi'(w^T x) \cdot x$
Unipolar: $\Phi' = y(1-y)$
Bipolar: $\Phi' = \frac{1}{2}(1-y^2)$

Common Mistakes to Avoid

1. In **Perceptron**, after computing I = $w^T x$, you MUST apply sgn before computing r. The y used in the error is ±1, not the raw dot product value.

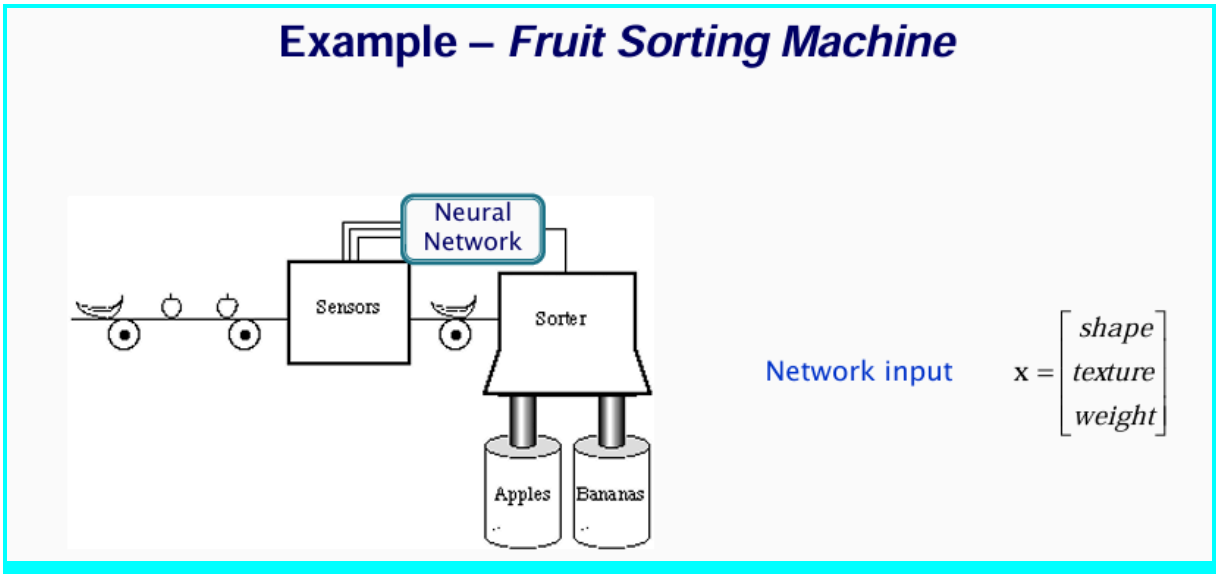
2. In **ADALINE**, y is the raw $w^T x$ value — do NOT apply sgn. This is why the weight change is often small and gradual.

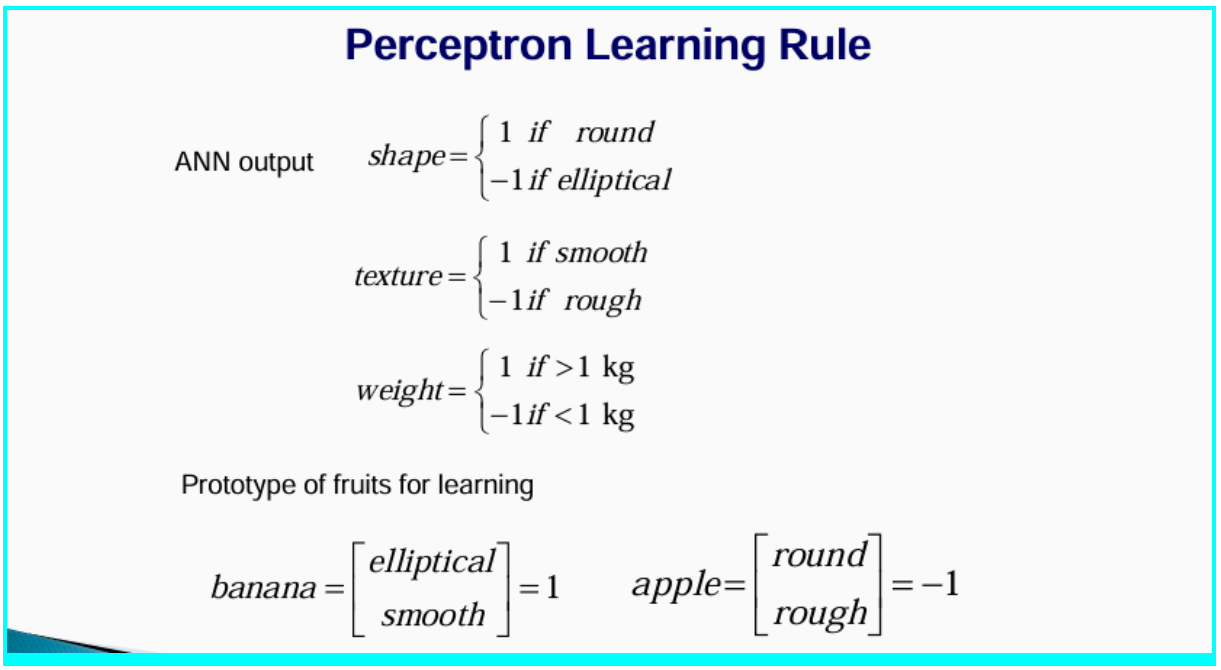
3. In **Delta**, compute I = $w^T x$ first, then y = Φ(I), then Φ'(I) = y(1−y) for unipolar. Do NOT forget the Φ'(I) term — this is what makes Delta different from ADALINE.

4. When r = 0 (desired = actual), Δw = 0 always, regardless of the rule. Skip the update.

5. Always use the **updated** weights for the next pattern, not the original ones.

Exam-e jodi scenario dey like "a fruit sorting machine receives inputs of shape, texture, weight, and size for a banana with values [−1, 1, 1, −1] and desired output +1, initial weights [0.5, −0.5, 0.5, −0.5], η=0.1, use perceptron rule" — then just extract those numbers and plug them into the steps above. The scenario is just decoration around the math.





ফ্রুট সর্টিং মেশিন — Perceptron Learning Rule

সিস্টেমটা কী করে?

একটা কনভেয়ার বেল্টে ফল আসছে। Sensor সেই ফলের তিনটা বৈশিষ্ট্য মাপে এবং Neural Network সিদ্ধান্ত নেয় এটা Apple নাকি Banana।

ইনপুট ভেক্টর

প্রতিটা ফলের জন্য তিনটা ইনপুট নেওয়া হয়:

$x = [\text{shape}, \text{texture}, \text{weight}]^T$

এনকোডিং — সংখ্যায় রূপান্তর

নেটওয়ার্ক শুধু সংখ্যা বোঝে, তাই বৈশিষ্ট্যগুলো এভাবে কনভার্ট করা হয়:

Shape (আকৃতি):

- গোলাকার (round) → **+1**
- লম্বাটে (elliptical) → **−1**

Texture (গায়ের ধরন):

- মসৃণ (smooth) → **+1**
- খসখসে (rough) → **−1**

Weight (ওজন):

- ১ কেজির বেশি → **+1**
- ১ কেজির কম → **−1**

প্রোটোটাইপ — আদর্শ ফলের মান

Banana:

- আকৃতি = elliptical = **−1**
- গায়ের ধরন = smooth = **+1**
- তাই $x(\text{banana}) = [-1, 1]^T$ → desired output **d = +1**

Apple:

- আকৃতি = round = **+1**
- গায়ের ধরন = rough = **−1**
- তাই $x(\text{apple}) = [1, -1]^T$ → desired output **d = −1**

সহজ ভাষায় সারকথা

> একটা ফল বেল্টে আসলে Sensor তার shape, texture, weight মেপে [+1 বা −1] আকারে Neural Network-এ পাঠায়। Network Perceptron rule দিয়ে শিখে নেয় কোন combination মানে Banana (+1) আর কোনটা Apple (−1) — এবং Sorter সেই অনুযায়ী ফলটাকে সঠিক বাক্সে ফেলে দেয়।

১. Unipolar Sigmoid Function

এটি একটি সাধারণ সিগময়েড ফাংশন যা আউটপুটকে 0 **থেকে** 1 এর মধ্যে সীমাবদ্ধ রাখে।

- **Function:**

$$\Phi(I) = \frac{1}{1 + e^{-I}}$$

- **Output Range:** $(0, 1)$
- **Derivative:**
  এখানে যদি আমরা আউটপুটকে $y = \Phi(I)$ ধরি, তবে এর ডেরিভেটিভ হবে:

$$\Phi'(I) = y(1 - y)$$

কেন এটি জনপ্রিয়? এর ডেরিভেটিভ বের করা খুব সহজ এবং ক্লিন, যা ক্যালকুলেশন স্পিড বাড়িয়ে দেয়।

২. Bipolar Sigmoid Function

এটি ইনপুটকে -1 **থেকে** $+1$ এর মধ্যে স্কোয়াশ (squash) করে। এটি শূন্য-কেন্দ্রিক (Zero-centered) হওয়ায় নিউরাল নেটওয়ার্ক দ্রুত শিখতে পারে।

- **Function:**

$$\Phi(I) = \frac{2}{1 + e^{-I}} - 1$$

- **Output Range:** $(-1, +1)$
- **Derivative:**
  এখানে যদি আউটপুট $y = \Phi(I)$ হয়, তবে ডেরিভেটিভ হবে:

$$\Phi'(I) = \frac{1}{2}(1 - y^2)$$

আমরা এই লার্নিং রুলগুলো সিরিয়ালি শিখি কারণ প্রতিটি নতুন রুল আগের রুলের কোনো না কোনো বড় সীমাবদ্ধতা (Limitation) দূর করে তৈরি হয়েছে। একে বলা হয় **Evolution of Neural Networks**।

নিচে সহজভাবে বোঝানো হলো কোনটি থেকে কোনটি কেন উন্নত:

---

১. Perceptron Learning Rule (শুরুটা যেখান থেকে)

এটি হলো একদম বেসিক রুল। এটি শুধু **Step Function** (আউটপুট শুধু ০ বা ১) নিয়ে কাজ করে।

- সীমাবদ্ধতা: এটি কেবল তখনই কাজ করে যদি ডাটা **Linearly Separable** হয় (অর্থাৎ সোজা দাগ দিয়ে আলাদা করা যায়)। যদি ডাটা একটুও এদিক-সেদিক থাকে, এই রুলটি কখনো থামে না (Converge করে না), চলতেই থাকে।
- পদ্ধতি: ভুল হলে ওয়েট বদলাও, ঠিক থাকলে কিছু করার দরকার নেই।

২. Delta Learning Rule (উন্নত গাণিতিক রূপ)

পারসেপট্রনের সীমাবদ্ধতা দূর করতে ডেল্টা রুল আসে। এটি **Gradient Descent** নামক একটি শক্তিশালী গাণিতিক ধারণা ব্যবহার করে।

- কেন উন্নত**?** * এটি শুধু ০ বা ১ দেখে না, বরং **Error-**এর পরিমাণ কতটুকু তা হিসাব করে (যেমন: $Target - Output$)।
    - এটি সব সময় ভুলের পরিমাণ কমানোর চেষ্টা করে (Error Minimization)।
    - ডাটা পুরোপুরি লিনিয়ারলি সেপারেবল না হলেও এটি কাছাকাছি একটি সেরা সমাধানে পৌঁছাতে পারে।

৩. Adaline Learning Rule (ADAptive LINear Element)

অ্যাডালাইন হলো ডেল্টা রুলের একটি বাস্তব প্রয়োগ। বার্নার্ড উইড্রো এটি তৈরি করেছিলেন।

- সবচেয়ে বড় পার্থক্য ও আপগ্রেড**:**
    - **Perceptron:** এখানে থ্রেশহোল্ড বা অ্যাক্টিভেশন ফাংশনের পরের আউটপুট দেখে ওয়েট আপডেট করা হয়।
    - **Adaline:** এখানে অ্যাক্টিভেশন ফাংশনের আগে (অর্থাৎ লিনিয়ার সামেশনের সময়) যে আউটপুট আসে, তা দেখে ওয়েট আপডেট করা হয়।
    - সুবিধা**:** এর ফলে এটি অনেক বেশি নিখুঁতভাবে বুঝতে পারে ওয়েট কতটুকু কমানো বা বাড়ানো দরকার। এটি অনেক বেশি 'Stable' বা স্থির।

সংক্ষেপে আপগ্রেড হিস্ট্রি**:**

রুল	মূল লক্ষ্য	আপগ্রেড বা বিশেষত্ব
Perceptron	শুধু শ্রেণীবিভাগ (Classification) করা।	অতি সাধারণ, ডাটা জটিল হলে ফেইল করে।
Delta Rule	Error কমানো (Least Mean Square)।	ক্যালকুলাস ব্যবহার করে Error মিনিমাইজ করা শেখে।
Adaline	নিখুঁত ট্রেনিং (Continuous values)।	লিনিয়ার আউটপুট ব্যবহার করে আরও স্থিতিশীল লার্নিং নিশ্চিত করে।

আমরা কেন এগুলো শিখি?

বর্তমানে আমরা যে **Backpropagation** (যেকোনো জটিল AI-এর প্রাণ) ব্যবহার করি, সেটি মূলত এই **Delta Rule**-এরই একটি মাল্টি-লেয়ার ভার্সন।

- **Perceptron** না বুঝলে তুমি নিউরাল নেটওয়ার্কের গঠন বুঝবে না।
- **Adaline/Delta Rule** না বুঝলে তুমি বুঝবে না কীভাবে গণিত ব্যবহার করে একটি মেশিন তার ভুল কমাতে পারে (Gradient Descent)।

সহজ কথায়, সিঁড়ির প্রথম ধাপ (Perceptron) না চড়লে তুমি শেষ ধাপে (Deep Learning) পৌঁছাতে পারবে না।